

React.js入门

简介

React 起源于 Facebook 的内部项目，因为该公司对市场上所有JavaScript框架，都不满意，就决定自己写一套，用来架设 Instagram 的网站。做出来以后，发现这套东西很好用，就在2013年5月开源了。由于 React 的设计思想极其独特，属于革命性创新，性能出众，代码逻辑却非常简单。所以，越来越多的人开始关注和使用，认为它可能是将来 Web 开发的主流工具。

环境配置

Next.js是React.js的一个生产级框架，请大家根据以下步骤创建一个Next.js项目
在你希望创建项目的目录下在终端中输入

```
npx create-next-app@latest
```

```
Dell@atri MINGW64 /h/desktop/repo/新建文件夹
$ npx create-next-app@latest
Need to install the following packages:
create-next-app@14.2.15
Ok to proceed? (y) y

✓ What is your project named? ... zjuqsc
✓ Would you like to use TypeScript? ... No / Yes
✓ Would you like to use ESLint? ... No / Yes
✓ Would you like to use Tailwind CSS? ... No / Yes
✓ Would you like to use `src/` directory? ... No / Yes
✓ Would you like to use App Router? (recommended) ... No / Yes
✓ Would you like to customize the default import alias (@/*)? ... No / Yes
```

输入项目名称，将TypeScript项选为Yes，其余按默认即可。
进入项目目录，打开终端输入

```
npm run dev
```

```
PS H:\desktop\repo\新建文件夹\zjuqsc> npm run dev

> zjuqsc@0.1.0 dev
> next dev

  ▲ Next.js 14.2.15
  - Local:      http://localhost:3000

  ✓ Starting...
  ✓ Ready in 1942ms
```

打开Local后的网址（按住ctrl再用鼠标左键点击即可），如果能显示出网页则配置成功（注意不要关闭终端）

NEXT.js

1. Get started by editing `app/page.tsx`.
2. Save and see your changes instantly.

[▲ Deploy now](#)[Read our docs](#)

[📖 Learn](#) [🔗 Examples](#) [🌐 Go to nextjs.org →](#)

自闭合标签

在HTML中存在一种简写形式，即自闭合标签。在 HTML 中，自闭合标签通常用于那些不需要包裹任何内容的元素。

```

```

修改网页名称等

Next框架中的元数据在 `/app/layout.tsx` 下

```

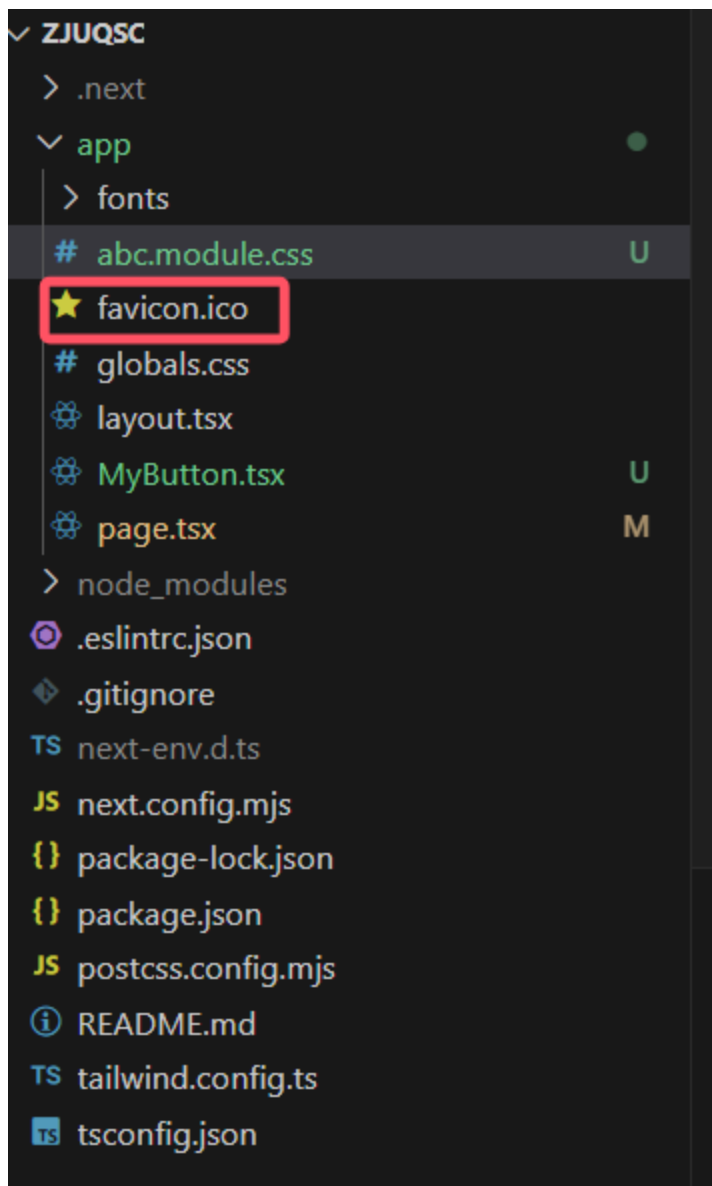
1  import type { Metadata } from "next";
2  import localFont from "next/font/local";
3  import "./globals.css";
4
5  const geistSans = localFont({
6    src: "./fonts/GeistVF.woff",
7    variable: "--font-geist-sans",
8    weight: "100 900",
9  });
10 const geistMono = localFont({
11   src: "./fonts/GeistMonoVF.woff",
12   variable: "--font-geist-mono",
13   weight: "100 900",
14 });
15
16 export const metadata: Metadata = {
17   title: "Create Next App",
18   description: "Generated by create next app",
19 };
20
21 export default function RootLayout({
22   children,
23 }: Readonly<{
24   children: React.ReactNode;
25 }>) {
26   return (

```

在这里修改即可

网页图标则直接替换

/app/favicon.ico 即可



创建和嵌套组件

Next.js框架中 `/app/page.tsx` 是打开 `http://localhost:3000/` 后默认显示的内容，修改该文件即可。

React中一切皆组件。一个组件是 UI（用户界面）的一部分，它拥有自己的逻辑和外观。组件可以小到 一个按钮，也可以大到整个页面。

React 组件是返回标签的 js/ts 函数：

```
function MyButton() {  
  return (  
    <button>I'm a button</button>  
  );  
}
```

至此，你已经声明了 MyButton 组件，我们可以把它嵌套到另一个组件中：

```
export default function MyApp() {
  return (
    <div>
      <h1>Welcome to my app</h1>
      <MyButton />
    </div>
  );
}
```

在 page.tsx 文件中，带有 export default 关键字的function的返回值即为该页面显示的内容。

注意到 <MyButton /> 是以大写字母开头的。React 据此识别 React 组件，因此**React 组件必须以大写字母开头**，而 HTML 标签则必须是小写字母。

也可以将组件分离出来写在一个文件中，并在另一个组件中对其进行引用，此时被引用的组件也应该有 export default 关键字

```
import MyButton from "./MyButton"
```

export 和 default 关键字的具体用法，感兴趣的同学可以自行了解。

使用 JSX/TSX 编写标签

React使用被称为JSX/TSX(JavaScript and XML)的标签语法来开发。（下面都称为TSX）

TSX 比 HTML 更加严格。你必须闭合标签，如
。你的组件也不能返回多个 TSX 标签。你必须将它们包裹到一个共享的父级中，比如 <div>...</div> 或使用空的 <>...</> 包裹。

p.s.空标签是一个虚拟的标签，它在网页中不会变成任何一种元素，也不能为其添加任何属性

添加样式

在 React 中，你可以使用 className 来指定一个 CSS 的 class。它与 HTML 的 class 属性的工作方式相同：

```
<img className="avatar" />
```

然后，你可以在一个单独的 CSS 文件中为它编写 CSS 规则。在文件开头引用即可

```
import './style.css'
```

- /app/global.css 在框架中会默认应用于全局

显示数据

TSX 会让你把标签放到 TypeScript 中。而大括号会让你“回到”TypeScript 中，这样你就可以从你的代码中嵌入一些变量并展示给用户。

```
type user = {
  name: string,
  age: number,
  imageUrl: string,
  imageSize: number[]
}

const user1: user = {
  name = 'ATRI',
  age = '14',
  imageUrl = 'https://baidu.com',
  imageSize = [200,200]
}

export default function Name(){
  return <div>
    {user1.name}
  </div>

  <img
    className="avatar"
    src={user1.imageUrl}
    alt={'Photo of ' + user1.name}
    style={{
      width: user1.imageSize[0],
      height: user1.imageSize[1]
    }}
  />
}
```

传递参数

父组件可以传递参数给子组件，即子组件可以读取其属性
此时函数接收的参数必须为对象，通常为 props

```

type person = {
  name: string,
  imageId: string
}

function getImageUrl(person: person, size: number) {
  return (
    'https://i.imgur.com/' +
    person.imageId +
    size +
    '.jpg'
  );
}

function Avatar(props:{person: person,size: number}) {
  return (
    <img
      className="avatar"
      src={getImageUrl(person)}
      alt={props.person.name}
      width={props.size}
      height={props.size}
    />
  );
}

export default function Profile() {
  return (
    <div>
      <Avatar
        size={100}
        person={{
          name: 'Katsuko Saruhashi',
          imageId: 'Yfe0qp2'
        }}
      />
      <Avatar
        size={80}
        person={{
          name: 'Aklilu Lemma',
          imageId: 'OKS67lh'
        }}
      />
    </div>
  );
}

```

```

    <Avatar
      size={50}
      person={{
        name: 'Lin Lanying',
        imageId: '1bX5QH6'
      }}
    />
  </div>
);
}

```

我们也可以选择属性中的几项进行传递

```

function Avatar({person,size}:{person: person,size: number}) {
  return (
    <img
      className="avatar"
      src={getImageUrl(person)}
      alt={person.name}
      width={size}
      height={size}
    />
  );
}

```

children

将嵌套在 TSX 标签中时，父组件将在名为 children 的 prop 中接收到该内容。例如，下面的 Card 组件将接收一个被设为 children 的 prop 并将其包裹在 div 中渲染：

```
export default function Profile() {
  return (
    <Card>
      <Avatar
        size={1000}
        person={{
          name: 'ATRI',
        }}
      />
    </Card>
  );
}

function Card({ children }: { children: any }) {
  return (
    <div className="card">
      {children}
    </div>
  );
}

type Person = {
  name: string
}

function Avatar({ person, size }: { person: Person, size: number }) {
  return (
    <img
      className="avatar"
      src={'https://img.picui.cn/free/2024/10/10/6707252485ee0.jpg'}
      alt={person.name}
      width={size}
      height={size}
    />
  );
}
```

响应事件

```
export default function Button() {  
  function handleClick() {  
    alert('你点击了我! ');  
  }  
  
  return (  
    <button onClick={handleClick}>  
      点我  
    </button>  
  );  
}
```

注意：应该传递函数本身，而不是调用函数。

useState

基本用法

```
export default function Button() {  
  let index: number = 0;  
  
  function handleClick(): void {  
    index = index + 1;  
  }  
  
  return (  
    <>  
      <button onClick={handleClick}>  
        Next  
      </button>  
      <div>  
        {index+1}  
      </div>  
    </>  
  );  
}
```

*Next.js框架会直接报错，因为其默认渲染机制为服务器端渲染，由于 HTML 内容是由服务器预先渲染的，用户在浏览器中可以更快地看到完整的页面内容。但是显然，需要与用户交互的组件是无法仅在服务器端渲染的，所以我们需要在写有含交互的组件的文件开头写

```
'use client';
```

接着我们点击按钮，发现竟然没有任何反应。

存在两个原因使得变化不可见：

1. 局部变量无法在多次渲染中持久保存。当 React 再次渲染这个组件时，它会从头开始渲染——不会考虑之前对局部变量的任何更改。
2. 更改局部变量不会触发渲染。React 没有意识到它需要使用新数据再次渲染组件。

要使用新数据更新组件，需要做两件事：

1. 保留渲染之间的数据。
2. 触发 React 使用新数据渲染组件（重新渲染）。

useState 就是为了解决这个问题诞生的。

要使用 useState，首先要在文件顶部导入

```
import { useState } from 'react';
```

然后将

```
let index = 0;
```

修改为

```
const [index, setIndex] = useState(0);
```

index 是一个 state 变量，setIndex 是对应的 setter 函数，即修改 index 所用的函数。后面括号内的值即为 index 变量的初始值。用这个函数修改相当于告诉 React：记住变量并重新渲染这个组件

使用方法为

```
function handleClick() {  
  setIndex(index + 1);  
}
```

我们会发现数字开始增加了

如果你渲染同一个组件两次，每个副本都会有完全隔离的 state！ 改变其中一个不会影响另一个。

批处理

```
import { useState } from 'react';

export default function Counter() {
  const [number, setNumber] :[number, (number:number) => void]= useState(0);
  // 这里不做类型注释也不会报错

  return (
    <>
      <h1>{number}</h1>
      <button onClick={() => {
        setNumber(number + 1);
        setNumber(number + 1);
        setNumber(number + 1);
      }}>+3</button>
    </>
  )
}
```

你可能会认为点击“+3”按钮会使计数器递增三次

但是实际的调用是这样

```
setNumber(0 + 1);
setNumber(0 + 1);
setNumber(0 + 1);
```

React 会等到事件处理函数中的所有代码都运行完毕再处理你的 state 更新。

如果真的要多次改变状态，你可以像 `setNumber(n => n + 1)` 这样传入一个根据队列中的前一个 state 计算下一个 state 的函数，而不是像 `setNumber(number + 1)` 这样传入 下一个 state 值。

命名惯例

通常可以通过相应 state 变量的第一个字母来命名更新函数的参数：

```
setEnabled();
setLastName();
setFriendCount();
```

更新state中的对象

page.tsx

```
'use client'

import { useState } from 'react';
import './style.css'

export default function MovingDot() {
  const [position, setPosition] = useState({
    x: 0,
    y: 0,
  });
  return (
    <div
      className='box'
      onPointerMove={e => {
        position.x = e.clientX;
        position.y = e.clientY;
      }}
    >
      {123}
      <div
        className='dot'
        style={{ transform: `translate(${position.x}px, ${position.y}px)` }} />
    </div>
  );
}
```

style.css

```

.box{
  position: relative;
  width: 100vw;
  height: 100vh;
}

.dot{
  position: absolute;
  background-color: red;
  border-radius: 50%;
  left: -10px;
  top: -10px;
  width: 20px;
  height: 20px;
}

```

我们看到圆点并没有跟着鼠标指针运动

这是因为没有使用 state 的设置函数，React 并不知道对象已更改。所以 React 没有做出任何响应。通过使用 setPosition，你在告诉 React：

- 使用这个新的对象替换 position 的值
- 然后再次渲染这个组件

```

onPointerMove={e => {
  position.x = e.clientX;
  position.y = e.clientY;
}}

```

改为

```

onPointerMove={e => {
  setPosition({
    x: e.clientX,
    y: e.clientY
  });
}}

```

此时圆点就会跟着鼠标运动了。

*关于如何处理嵌套对象和数组，有兴趣的同学可以参考文档

[更新嵌套对象](#)

[更新数组](#)

思考题

修改下面这段代码，使其满足要求

1. 点击按钮会展开文字
2. 最多只有一段文字展开，即一段文字的展开会导致另一段关闭

page.tsx

```

import "./style.css"

export default function Accordion() {
  return (
    <>
      <div className='title'>Almaty, Kazakhstan</div>
      <Panel
        title="关于"
      >
        阿拉木图人口约200万，是哈萨克斯坦最大的城市。在1929年至1997年之间，它是该国首都。
      </Panel>
      <Panel
        title="词源"
      >
        这个名字源于哈萨克语 <span lang="kk-KZ">алма</span>，是“苹果”的意思，通常被翻译成“满是苹果”
      </Panel>
    </>
  );
}

type Panel = {
  title:string,
  children:any
}

function Panel({
  title,
  children
}:Panel) {
  return (
    <section className="panel">
      <h3>{title}</h3>
      <button>
        显示
      </button>
    </section>
  );
}

```

style.css (不需要对此进行改动)

```
.panel{
  border: 2px;
  border-color: black;
  border-style: solid;
  margin-top:1px;
}

button{
  border: 1px;
  border-color: black;
  border-style: solid;
  background-color: gray;
}

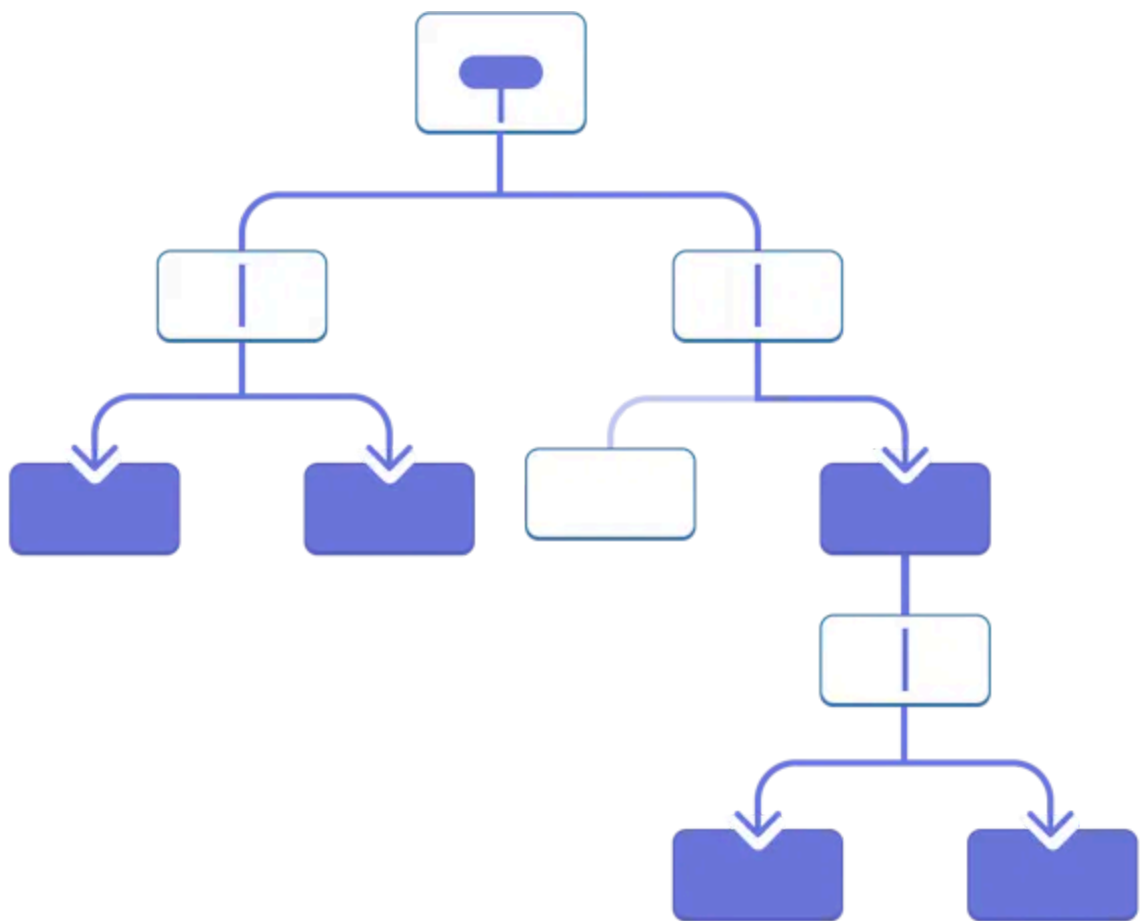
.title{
  text-align: center;
  font-size: 30px;
}

h3{
  font-size:25px
}
```

注：有时候，你希望两个组件的状态始终同步更改。要实现这一点，可以将相关 state 从这两个组件上移除，并把 state 放到它们的公共父级，再通过 props 将 state 传递给这两个组件。这被称为“状态提升”，这是编写 React 代码时常做的事。

使用 Context 深层传递参数

通常来说，你会通过 props 将信息从父组件传递到子组件。但是，如果你必须通过许多中间组件向下传递 props，或是在你应用中的许多组件需要相同的信息，传递 props 会变的十分冗长和不便。Context 允许父组件向其下层无论多深的任何组件提供信息，而无需通过 props 显式传递。



假设你想让相同 section 中的多个 Heading 具有相同的尺寸：

```

export default function Page() {
  return (
    <section>
      <Heading level={1}>主标题</Heading>
      <section>
        <Heading level={2}>副标题</Heading>
        <Heading level={2}>副标题</Heading>
        <Heading level={2}>副标题</Heading>
        <section>
          <Heading level={3}>子标题</Heading>
          <Heading level={3}>子标题</Heading>
          <Heading level={3}>子标题</Heading>
          <section>
            <Heading level={4}>子子标题</Heading>
            <Heading level={4}>子子标题</Heading>
            <Heading level={4}>子子标题</Heading>
          </section>
        </section>
      </section>
    </section>
  );
}

```

显然将 level 参数传递给 <section> 而不是传给 <Heading> 组件看起来更好一些。这样的话你希望使同一个 section 中的所有标题都有相同的尺寸：

```

<section level={3}>
  <Heading>关于</Heading>
  <Heading>照片</Heading>
  <Heading>视频</Heading>
</section>

```

1. 创建一个 context。你可以将其命名为 LevelContext, 因为它表示标题级别。
2. 在需要数据的组件内 使用 刚刚创建的 context。Heading 将会使用 LevelContext。
3. 在指定数据的组件中 提供 这个 context。section 将会提供 LevelContext。

Context 可以让父节点，甚至是很远的父节点都可以为其内部的整个组件树提供数据。

创建context

LevelContext.tsx

```
import { Context, createContext } from 'react';

export const LevelContext: Context<number> = createContext(0);
```

使用context

Next框架中服务器组件不允许使用 `createContext`
为了在组件中使用 `context` , 我们需要先导入

```
import { useContext } from 'react';
import { LevelContext } from './LevelContext.tsx';
```

使用 `useContext`

Heading.tsx

```
export default function Heading({ children }:any) {

  const level: number = useContext(LevelContext);
  switch (level) {
    case 1:
      return <h1>{children}</h1>;
    case 2:
      return <h2>{children}</h2>;
    case 3:
      return <h3>{children}</h3>;
    case 4:
      return <h4>{children}</h4>;
    case 5:
      return <h5>{children}</h5>;
    case 6:
      return <h6>{children}</h6>;
    default:
      throw Error('未知的 level: ' + level);
  }
}
```

提供context

Section.tsx

```
import { LevelContext } from './LevelContext.js';

export default function Section({ level, children }) {
  return (
    <section className="section">
      <LevelContext.Provider value={level}>
        {children}
      </LevelContext.Provider>
    </section>
  );
}
```

这告诉 React: “如果在 `<Section>` 组件中的任何子组件请求 `LevelContext`, 给他们这个 `level`。”组件会使用 UI 树中**在它上层最近**的那个 `<LevelContext.Provider>` 传递过来的值。

目前, 你仍需要手动指定每个 `section` 的 `level`。

由于 `context` 让你可以从上层的组件读取信息, 你可以让 `Section` 都会从上层的 `Section` 读取 `level`, 并自动向下层传递 `level + 1`。

```
export default function Section({ children }) {
  const level = useContext(LevelContext);
  return (
    <section className="section">
      <LevelContext.Provider value={level + 1}>
        {children}
      </LevelContext.Provider>
    </section>
  );
}
```

这样修改之后, 你不用将任何 `level` 参数传给 `<Section>` 或者是 `<Heading>` 了。

但是我们会注意到, 以上能做到的东西我们用 `props` 也能做到, 为什么一定要用 `context` 呢, 主要还是考虑到 `props` 逐级透传的问题

例如暗夜模式。